

Project - Closed loop DC motor with PID control

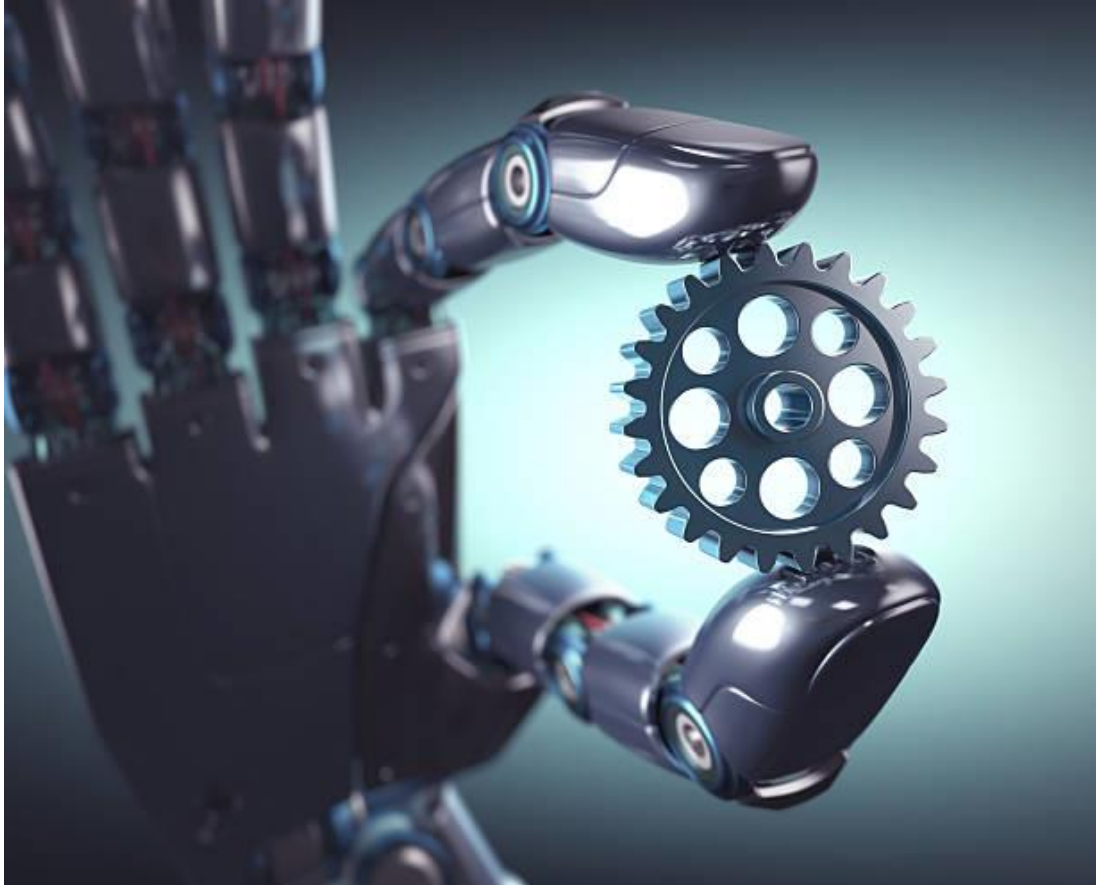


Table of contents

Project description	1
System requirements	2
Technical specifications.....	2
System design.....	3
Mathematical modelling of the DC motor	3
Design of PID controller.....	6
Simulation results.....	10
System implementation	12
Hardware implementation and testing	12
Implementation of speed sensor reading	13
Implementation of PID controller	14
Results	15
Conclusions.....	17
Appendix: Source codes	17

Project description

This project is about designing and implementing a digital closed loop DC motor control system with a PID-controller. The design and implementation of the control system must be in accordance with the project's system requirements. Design and implementation can be divided into four steps:

1. Mathematical modelling of DC motor
2. Design of PID-controller
3. Design algorithm for speed sensor reading
4. Implementation of digital PID controller

Step 1 and 2 will be done in Matlab and Simulink. For step 3 and 4 we will use C-programming for the Arduino ATmega2560 board.

This is a multidisciplinary project where knowledge from control theory, instrumentation, signal processing and communications, programming, analog and digital electronics, systems engineering and project management will be used.

System requirements

Table 1 shows the system requirements for the closed loop DC motor control system.

Steady-state speed	500 rpm
Steady-state error	10 %
Overshoot	Below 10 %
Rising time	10 s
Stability	Yes
Digital controller	PID

Table 1: System Requirements

Technical specifications

Table 2 shows the technical specifications for the hardware components in this project.

Microcontroller	Arduino ATmega 2560
Motor driver	L298N
Motor type	12 V permanent magnet motor with carbon brushes
Motor encoder	1024 PPR encoder with index
Motor size	4.2 cm x 3.2 cm x 8.9 cm
Motor weight	185 g

Table 2: Technical specifications for hardware components

System design

This section will cover the design process for the mathematical modelling of the DC motor and the design of the closed loop PID-controller.

Mathematical modelling of the DC motor

The model of the DC motor system is shown in figure 1. It consist of an armature and a load where R is the armature resistance, L is the armature inductance, v is the voltage applied, i is the current through the motor, e is the back emf voltage, J is the moment of inertia of the load, B is the viscous friction coefficient, τ is the torque generated by the motor, θ is the angular position of the motor, ω is the angular velocity and m is the external torque.

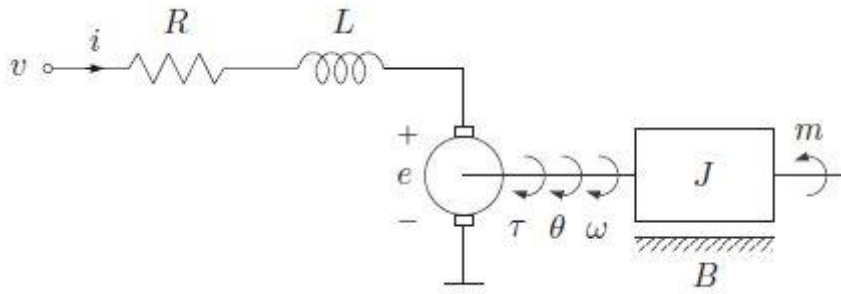


Figure 1: DC motor model

The back emf voltage can be expressed as: $e = K\omega$ where K is the back emf constant of the motor.

The motor torque can be expressed as: $\tau = K_i i = J\dot{\omega} + B\omega + m$ where K_i is the torque constant of the motor.

Applied voltage can be described by Kirchoff's voltage law: $v = L\dot{i} + Ri + K\omega$

The closed loop DC motor can be described mathematically by the transferfunction:

$$\frac{\Omega(s)}{V(s)} = \frac{K_i}{(Ls + R)(Js + B) + K\omega * K_i}$$

Where $\Omega(s)$ and $V(s)$ are laplace transforms of $\omega(t)$ and $V(t)$.

We can find the values for these parameters from the datasheet for our Zelcom DC motor. the parameters are as shown in table 3.

Parameter	Value
R	5.22 Ω
L	2.95 x 10 ⁻³ H
B	1.09 x 10 ⁻⁵ Nms
J	1.60 x 10 ⁻⁶ Nms ²
Kw	2.34 x 10 ⁻² Vs
Ki	2.34 x 10 ⁻² Nm/A

Table 3: DC motor parameters

By inserting these values into the transferfunction in Matlab, we can use different Matlab commands to obtain important information about the closed loop system.

Figure 2 shows the pole-zero diagram for our closed loop system.

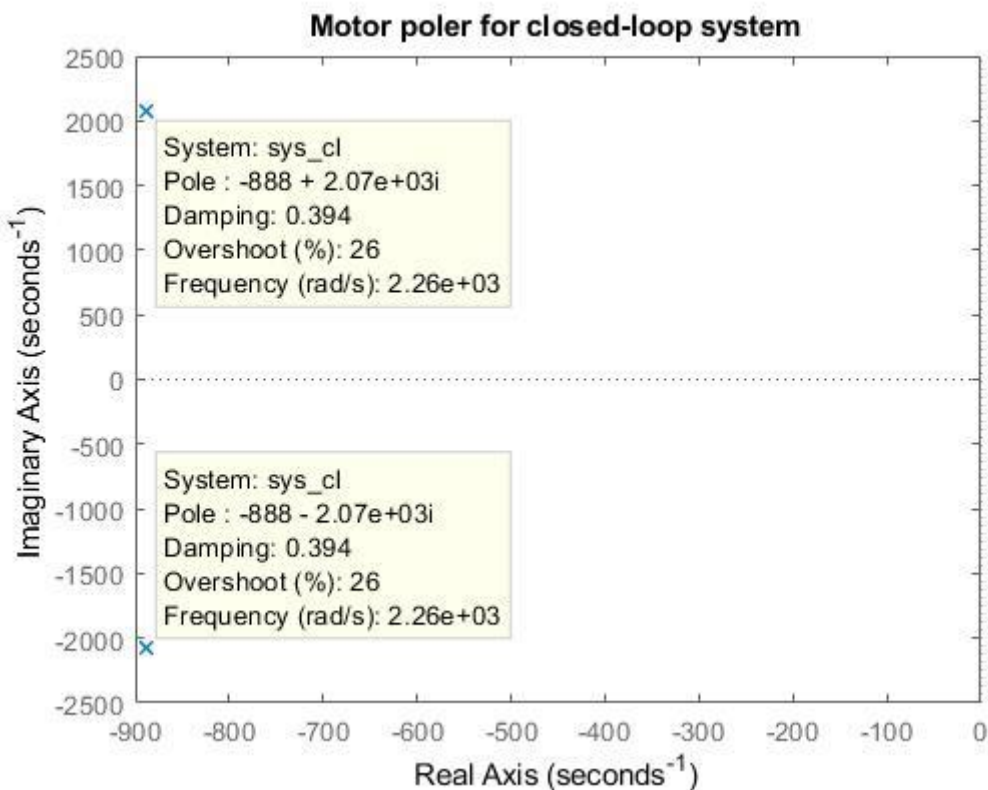


Figure 2: Pole zero-diagram for the closed loop system

Figure 3 shows the bode-diagram for our closed loop system.

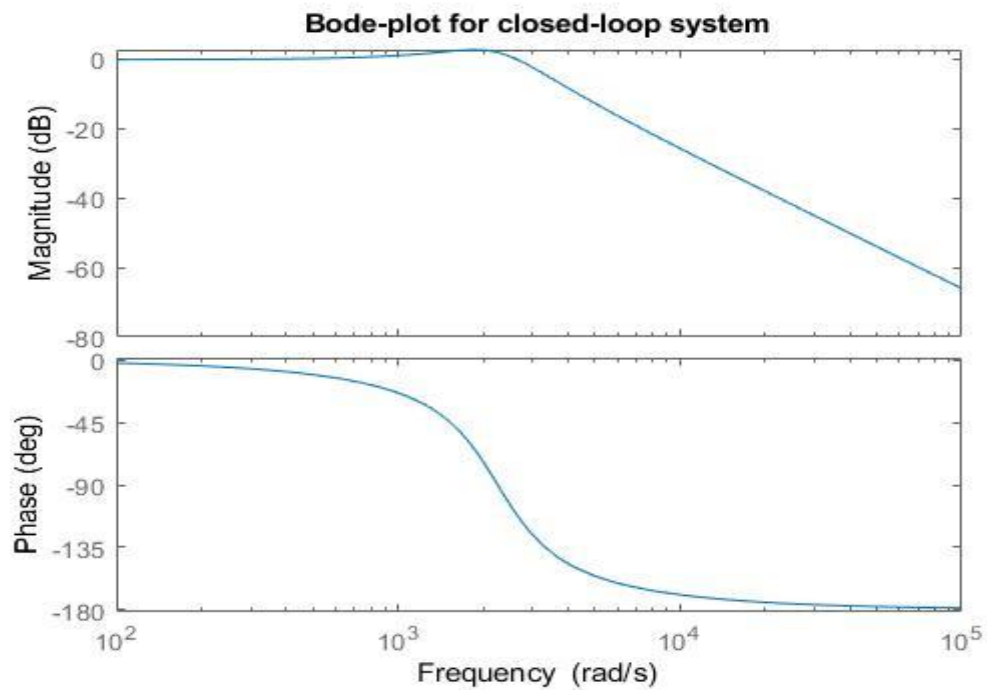


Figure 3: Bode-diagram for the closed loop system

Figure 4 shows the step-response for the closed loop system when we apply a step value that goes from 0 to 1 at the input.

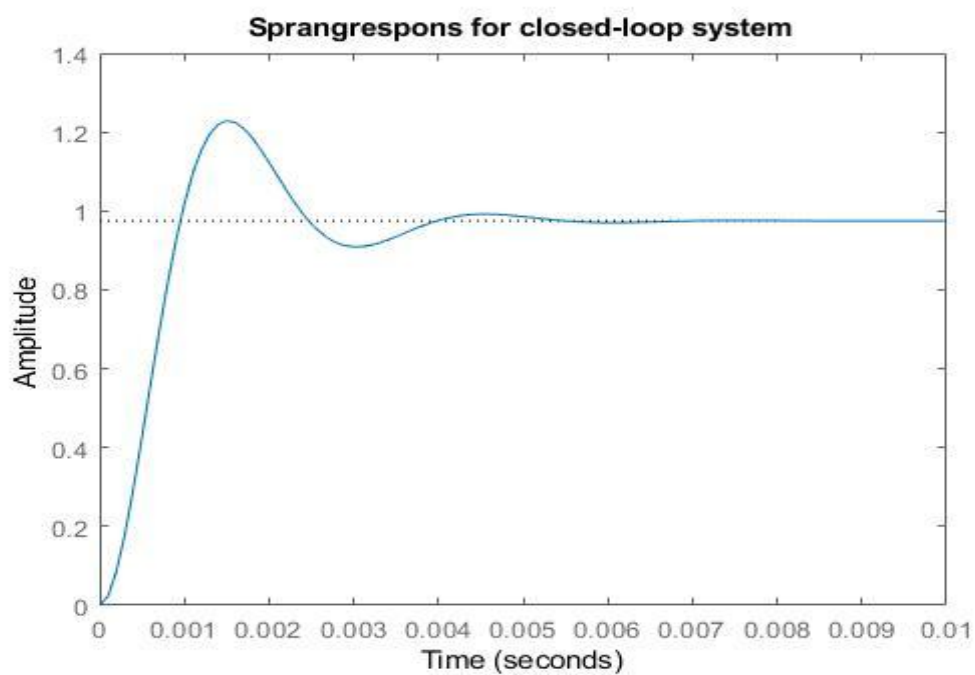


Figure 4: Step-response for the closed loop system

Design of PID controller

For our closed loop system to be able to match the system requirements, we will use a PID controller. Depending on how we tune the P, I and D parameters, we can modify the closed loop system's rise time, settling time overshoot and stability. For tuning of the PID controller we have two options, we can either experiment with inserting different PID values manually or use the built-in auto-tuning tool in Simulink, for this project we will use the latter.

In Simulink we first build a block diagram for the closed loop system with PID controller (see figure 5 and 6).

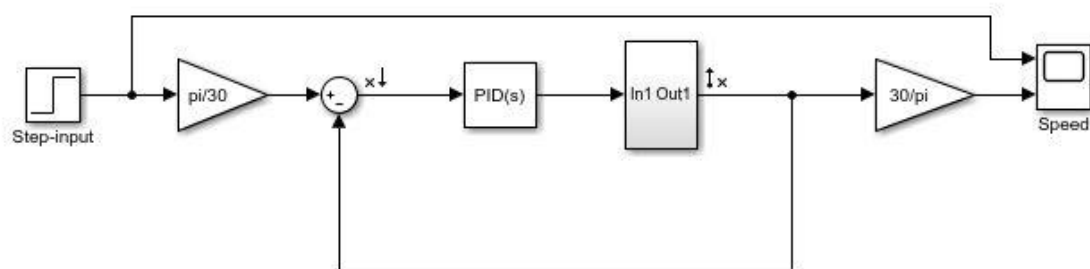


Figure 5: Overall block diagram of closed loop system with PID controller

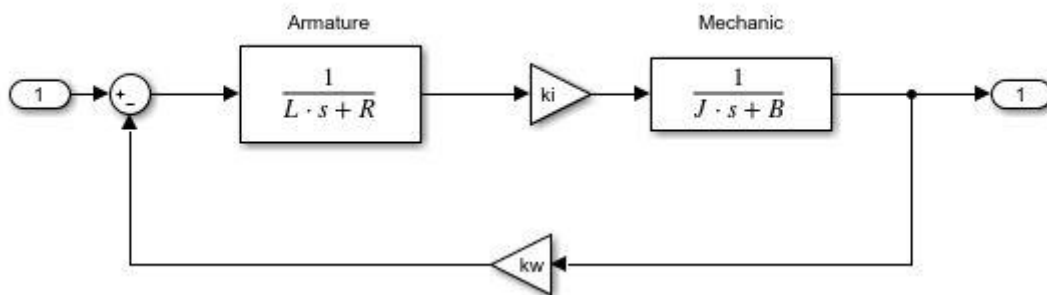


Figure 6: Block diagram for the DC motor model

By clicking on the PID block and select the “Tune” option a new window with the Simulink auto-tuning tool will be opened. By adjusting response time and transient behaviour, the program will automatically calculate the P, I and D values required for our wanted response (see figure 7).

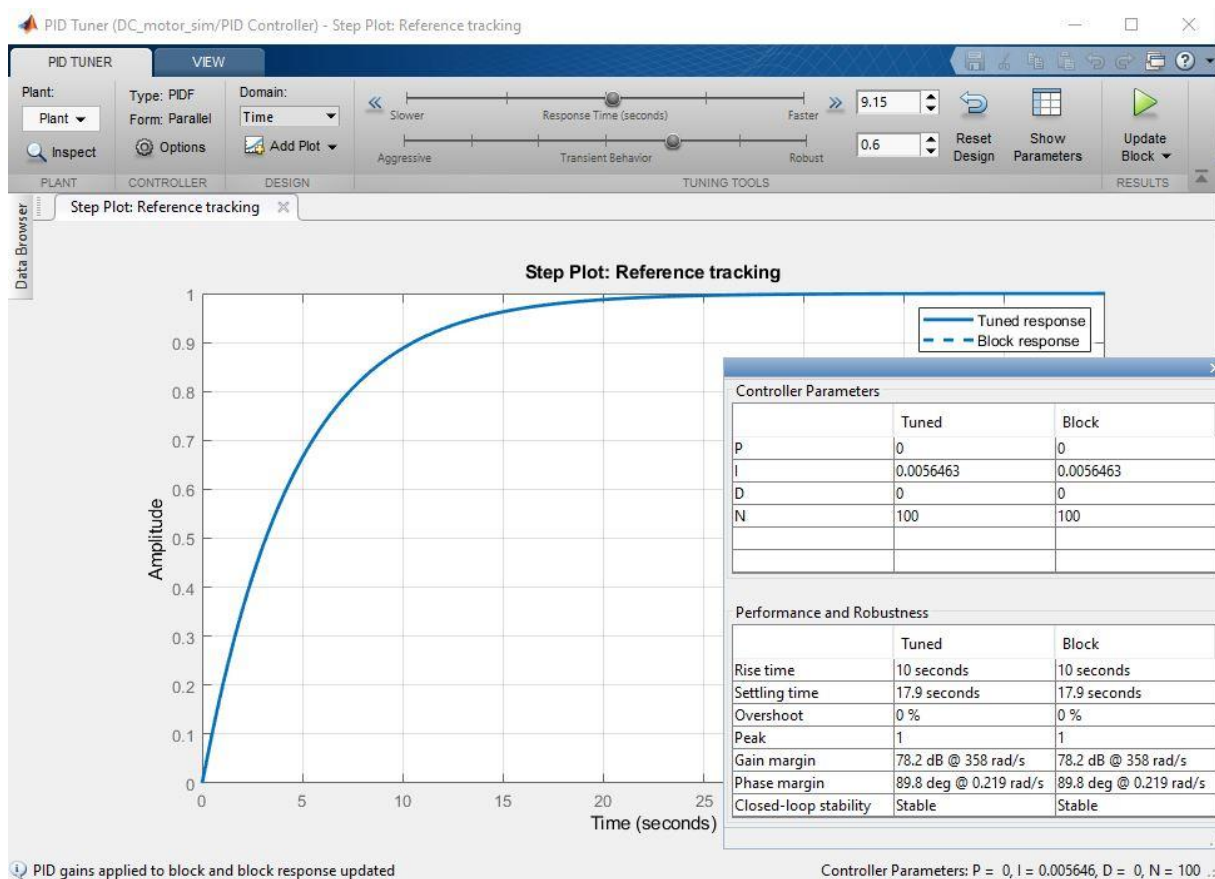


Figure 7: PID auto-tuning tool

Here we can see that to be able to get a rise time on 10 seconds, settling time on 17.9 seconds and a 0 % overshoot, we need to set the value of P parameter to 0, I parameter to 0.0056463 and D parameter to 0. But this Simulink model and tuning is for the analog version of the DC motor closed loop system.

Since the system requirements says that the system requires a digital controller, we need to digitalize the closed loop control system in Simulink. To digitalize the system means to reconstruct the analog response with discrete values, to be able to do that we need to determine the sampling frequency. We can determine the sampling frequency by looking at the frequency response of the analog control loop system as shown in figure 8.

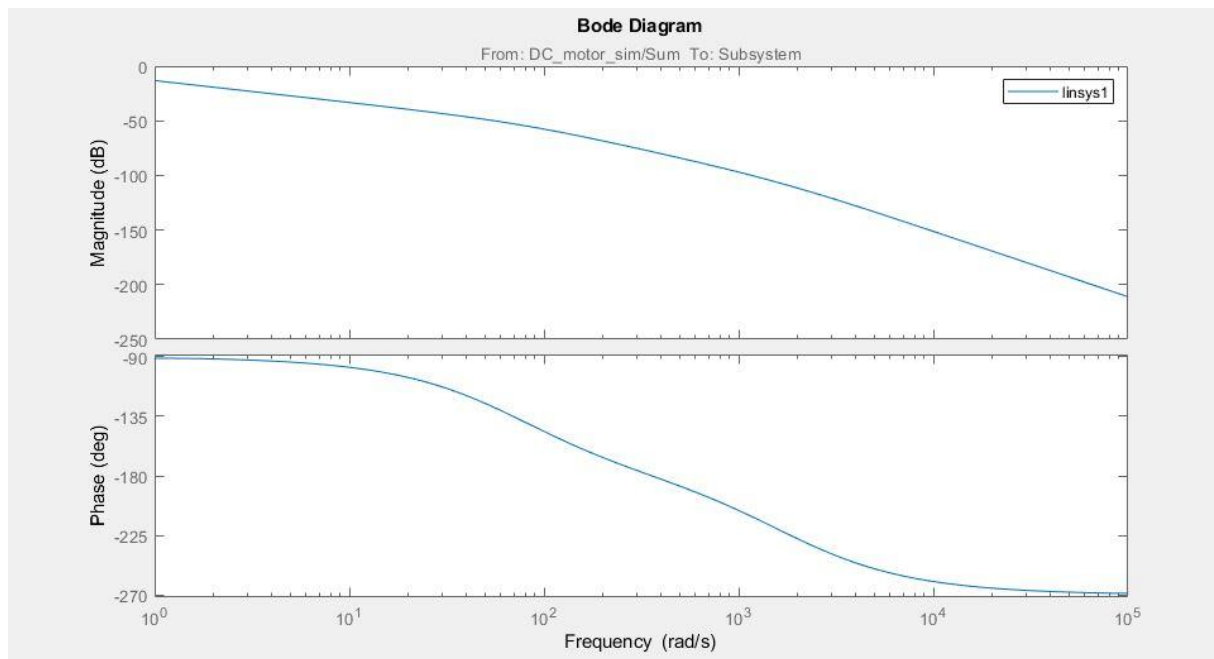


Figure 8: Bode diagram of closed loop system with PID controller

By zooming in on the Magnitude response graph and looking at the point that is -3 dB below the starting point, we can read the max frequency in rad/s (see figure 9).

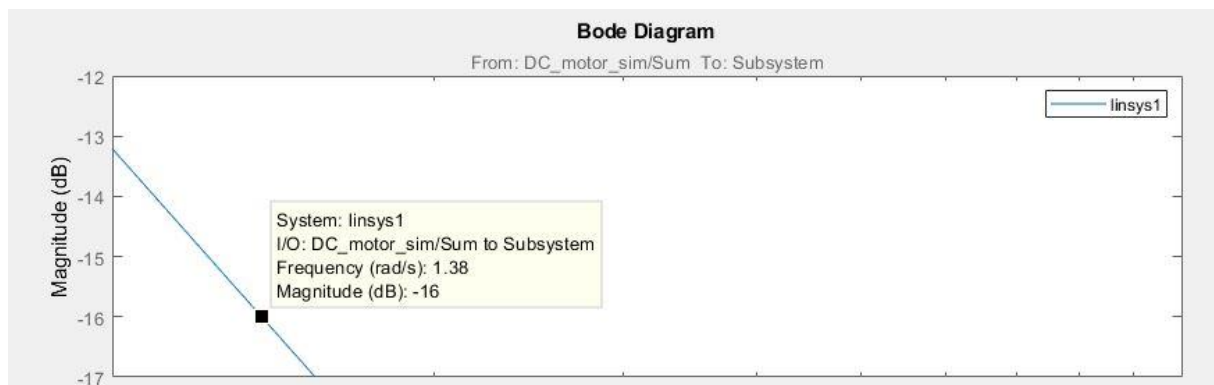


Figure 9: Zoomed in Magnitude response

The graph reads a max frequency of 1.38 rad/s. The Nyquist theorem says that in order to avoid aliasing, we need a sampling rate twice as large or greater as the largest frequency component in the analog signal. We convert the max frequency we found into Hertz:

$$f_{\max} = \frac{\omega}{2\pi} = \frac{1.38}{2\pi} = 0.219 \text{ Hz}$$

We find the sampling rate by using the Nyquist formula:

$$f_s \geq 2 * f_{\max} = 0.438 \text{ Hz}$$

We need a sampling frequency of atleast 0.438 Hz or greater, by choosing a sampling frequency of 2 Hz we get a sampling time of 0.5 s that should be good enough to reconstruct the analog signal.

Figure 9 shows the block diagram for the digitalized PID controller control system.

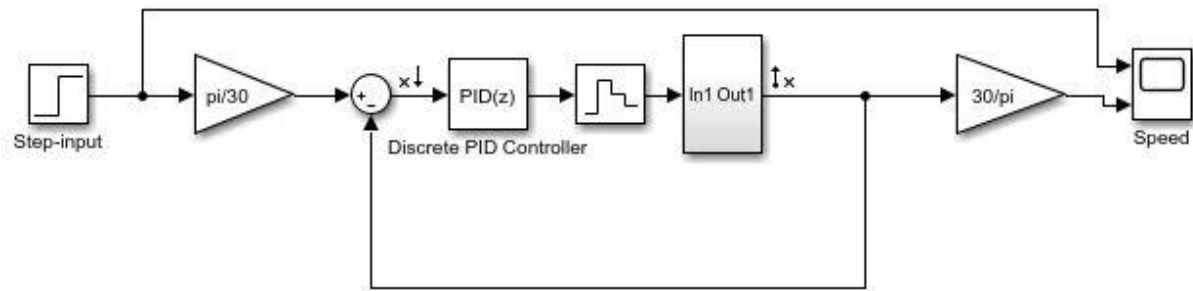


Figure 9: Digitalized closed loop system with PID control

In this block diagram we have included a zero-order hold block that holds the sampled value until the next sampling occurs. We repeat the process for PID tuning like we did with the analog version and get the results shown in figure 10.

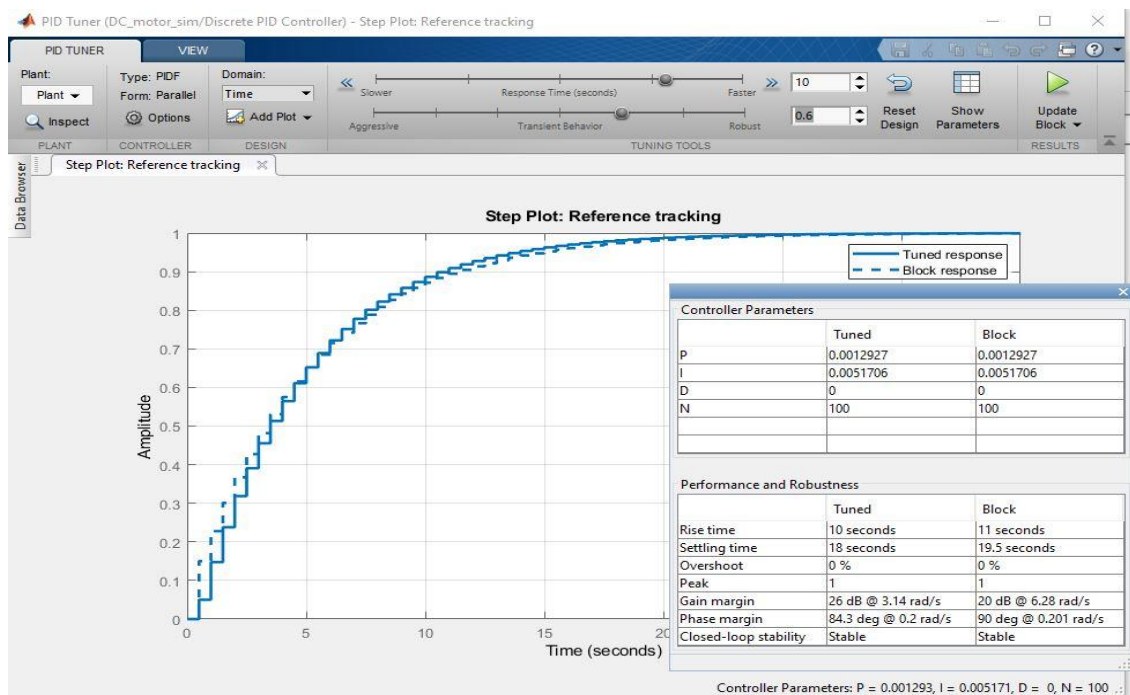


Figure 10: PID tuning for digitalized controller

From figure 10 we get a P parameter equal to 0.0012927, I parameter equal to 0.0051706 and D parameter equal to 0. This results in closed loop stability, rise time of 10 seconds, settling time of 18 seconds and 0 % overshoot. We will implement the digital controller with these PID parameters we found here.

Simulation results

We can verify our designs by running simulations on them in Simulink and test that the tuned PID parameters gives expected results. The simulations display the output as a function of time where time is on the x-axis and output value on the y-axis.

Figure 11 shows the results for the analog closed loop PID controller.

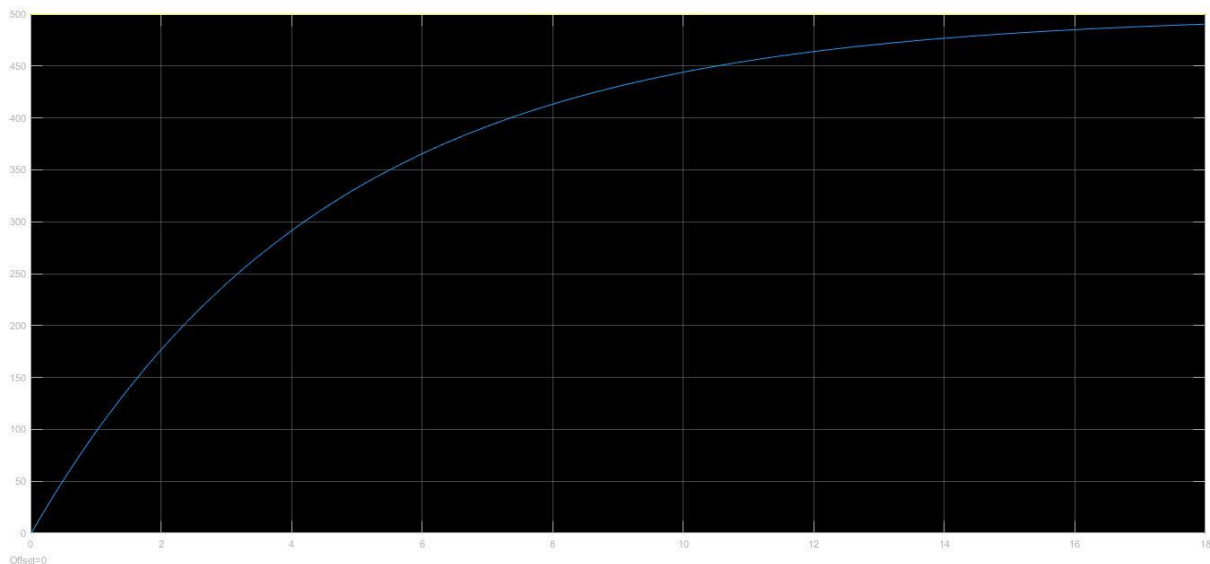


Figure 11: Analog closed loop PID controller

From the graph we can see that we have 10 seconds rising time and 18 seconds settling time with a slight steady-state error. In the requirements a 10 % steady-state error is allowed, so if the output don't go below 450 rpm or above 550 rpm it is fine.

Figure 12 shows the results for the digitalized closed loop PID controller.

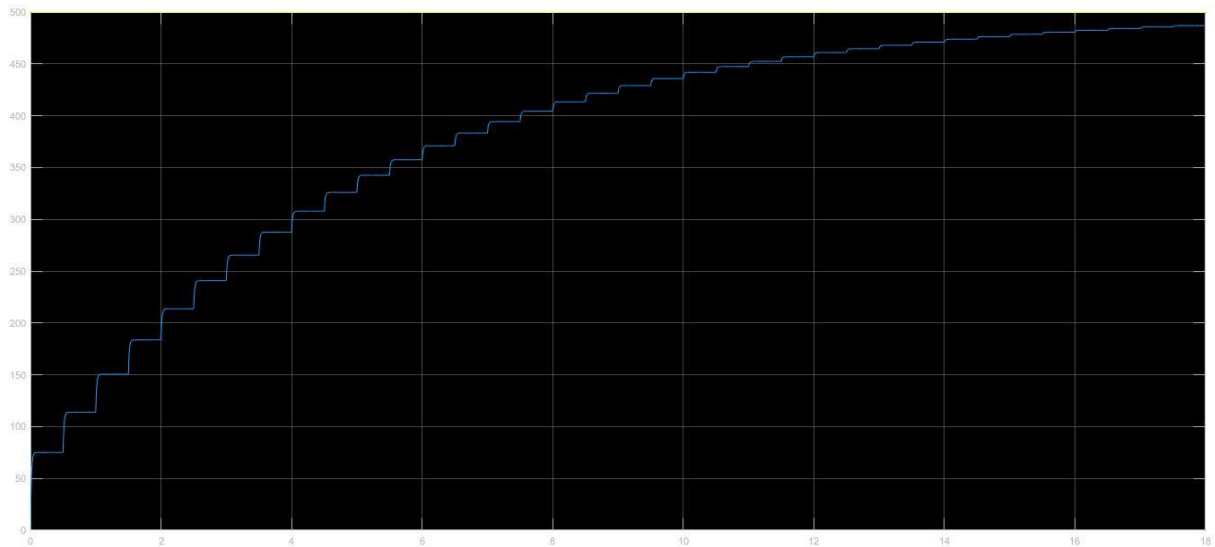


Figure 12: Digitalized closed loop PID controller

The simulation yields the same results regarding rise time, settling time and steady-state error. The only difference is the output form, here it has a ladder shaped form due to discrete sampling every 0.5 second. The zero-order hold function makes sure to hold the sampled value until the next sampling is triggered. From figure 12 we can see that the value “climbs” 4 times every second.

Figure 13 shows the analog and digital simulation results combined.

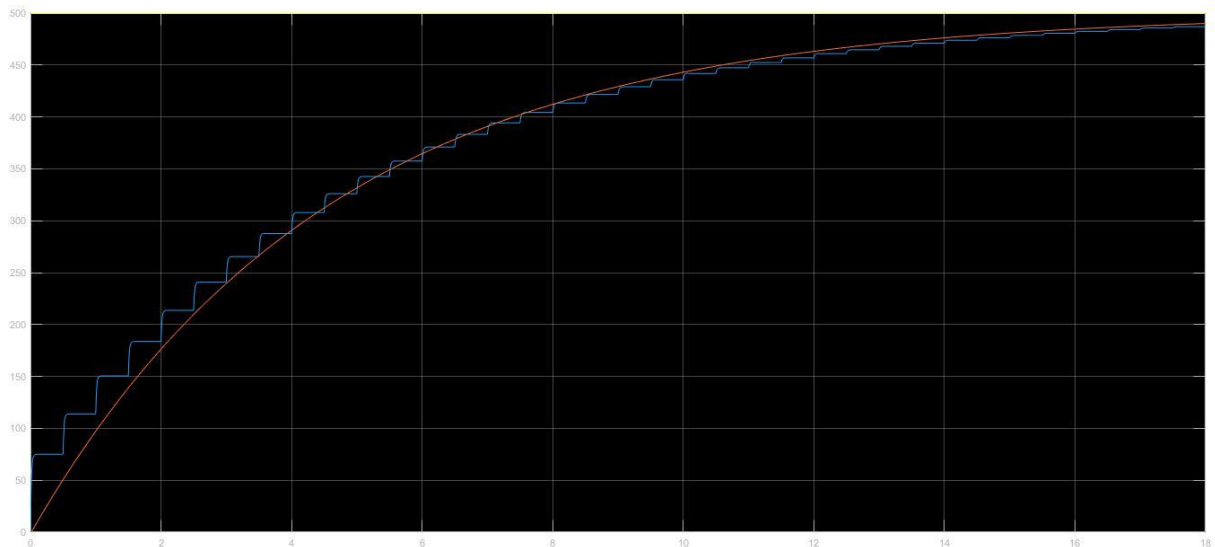


Figure 13: Combined simulation results

System implementation

This section will cover the implementation of the closed loop DC motor with PID controller in Arduino which includes implementation of the speed sensor reading and PID control. I decided to divide the system implementation into several smaller steps. Dividing the implementation into smaller steps will make it easier for testing and verification of system functionality along the way as complexity increases, and it will also make tracking of progress easier.

Hardware implementation and testing

To ensure that all the hardware components was working properly, I connected the hardware together as shown in figure 14 with a pushbutton and pot-meter included.

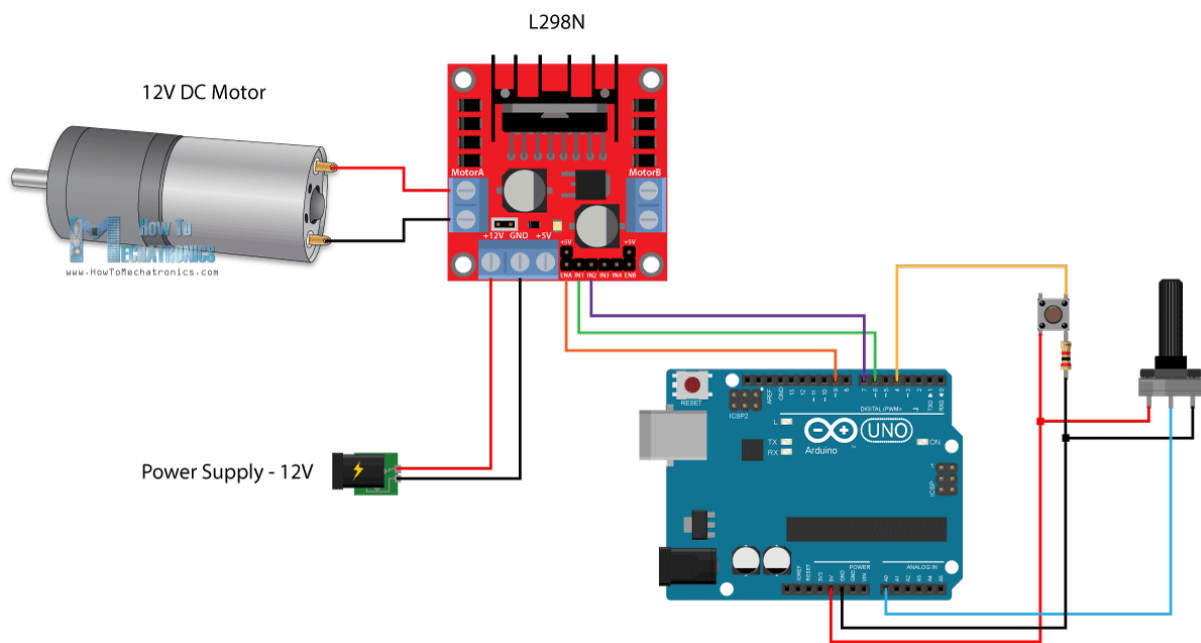


Figure 14: Wiring diagram for hardware testing

I implemented some simple Arduino code to test that I was able to manually control the DC motor through the L298N motor driver. I programmed the push button to change the rotational direction of the motor when pressed. I programmed the Arduino to read the analogue pot-meter values and map them to PWM values that was proportional to the pot-meter values. The PWM values was then sent to the motor driver to adjust the speed of the motor depending on what position the pot-meter was in. After confirming that all the hardware was working and that it was possible to manually control the DC motor, I moved on to the next step.

Implementation of speed sensor reading

To be able to read the DC motor speed, I connected the DC motor rotary encoder to the Arduino board. I connected the red wire to +5 V, black wire to ground and the green wire to digital pin 21 on the ATmega2560 board. The rotary encoder has 1024 points per round, that means that the encoder will output a high pulse to the Arduino whenever it detects a point from the rotating motor. After 1024 points has been counted, the motor has turned one revolution.

Digital pin 21 on the Arduino board is compatible with interrupt functionality. Interrupt functions are handy for functions you want to perform in the background that will not disturb the main functions of the program. In my program I declared a variable called “pulses” which I initialized to 0. I declared digital pin 21 to call a interrupt function named “count()” to increase the variable “pulses” by 1 every time the encoder sensor detects a rising pulse edge. A rising pulse edge means whenever the pulse goes from low to high, a point has been detected and the Arduino program will count the points as the motor rotates.

I tested and verified that the interrupt routine worked by printing the pulse variables to the serial console while increasing the speed with the pot-meter, the serial console outputted numbers that increased fast over time so the pulse counting works.

Now that we have a code that is able to count the points of the rotary encoder, we can implement an algorithm that takes the numbers of points counted within a certain timeframe, lets say 0.5 seconds since this is our defined sampling time. When we know the number of points per 0.5 second, we can easily convert that to numbers of points per minute by multiplying with 120, and if we divide that by 1024 we finally get what we are looking for, namely revolutions per minute (rpm). The final conversion formula is:

$$\text{RPM} = \frac{\text{Nr.of pulses} * 120}{1024}$$

The speed-reading algorithm design is described in these steps:

1. Main loop calls for speed-reading function
2. Speed-reading function samples the speed every 0.5 s
3. Speed-reading function saves the number of counts from the most recent 0.5 s timeframe
4. Speed-reading function subtracts the number of counts from the most recent timeframe with the number of counts from the previous timeframe.
5. Speed-reading function converts the result from step 4 to RPM by using the final conversion formula.
6. Speed-reading function saves the current result in another variable to be compared with the number of counts for the next sample.
7. Speed reading function prints the RPM value to serial console.

Source code for the speed-reading algorithm can be found in the appendix.

Implementation of PID controller

The digital PID controller will be the main part of our Arduino program. I removed the pushbutton and pot-meter from the hardware configuration, because now the PID controller will take care of the speed adjustments. We need to declare six variables for the PID controller function:

1. PID input will be declared as the measured RPM value from speed sensor.
2. PID output will be calculated by the PID controller and sent as PWM voltage to DC motor.
3. Setpoint will be declared as 500 rpm.
4. K_p will be declared as the proportional gain value
5. K_i will be declared as the integral gain value
6. K_d will be declared as the derivative gain value

The PID controller will measure the closed loop control system's error which can be described as:

$$e = \text{setpoint} - \text{input}$$

The PID controller will use the value of error in three separate calculations, for the P, I and D controller as shown in figure 15.

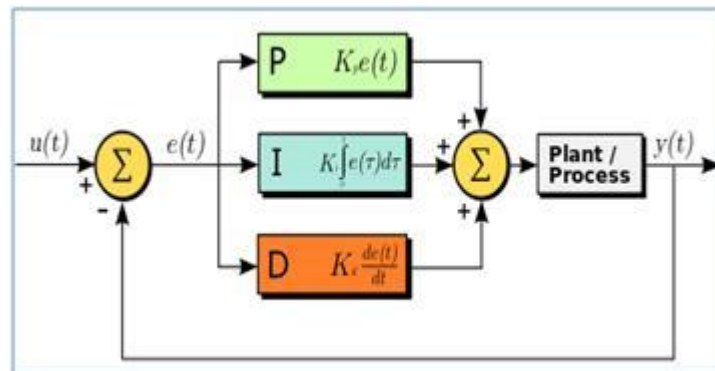


Figure 15: Illustration of PID controller calculations

The PID controller will multiply e as a function of time with the proportional gain constant, the integral of e with respect to time multiplied by the integral gain constant and lastly the derivative of e with respect to time with the derivative constant and add them all together. The sum of all 3 gains will be the output which in our case will be the PWM value that is sent to the DC motor to control the speed.

I first tried to write my own software algorithm to implement the PID controller and test it, but it resulted in a very unstable system with too high overshoots and the steady-state error was way over 10%. Due to time limitations I didn't see any reason to try to invent the wheel again, so I decided to use an existing Arduino PID library for the implementation of the PID controller. Source codes for both my own code and the PID library is included in the appendix.

My overall program is structured with a main loop, inside the main loop there is three functions that is called in sequence; measure speed, PID control and send PWM value to DC motor. The program first counts the pulses from the interrupt routine and converts it to speed measured in rpm, the rpm is then used as input for the PID controller function, the PID controller does the necessary calculations with the defined variables that is passed to the function and produces an output in the form of a PWM voltage to control the motor speed. I customized the library and set a sampling time of 0.5 seconds for the PID controller process to match the sampling rate we calculated. The PID controller will do a new calculation every half second and try to make the error value as small as possible so we get an output that matches the setpoint value.

Results

My code together with the PID-library works as intended. The DC motor reaches the desired setpoint value automatically with the closed loop feedback control. But I was not successful to fully reach all the system requirements. When I run the program, the DC motor stands still at zero RPM for a long time before it suddenly takes a huge jump to an rpm close to setpoint value. The response also varies every time I run the code, sometimes it jumps to 150 rpm, then 400 rpm before getting to the goal of 500 rpm. Other times it can give an overshoot over 10% before going down to setpoint value. When reached setpoint value, there is still some output "ripple" as the speed will have continuous small variations within the 10% steady-state error, but it doesn't reach fully a steady-state constant speed. The rising time is longer than 10 seconds, it takes like 18-20 seconds for the speed to be near setpoint value. If I use a faster sampling time for sensor reading and PID calculations the system seems to be slightly more stable, but still struggles to hold a constant steady-state speed. If I increase the setpoint value it seems to rise more gradually to setpoint value instead of taking those large jumps from still position to fast rotation. Figure 16 shows a test run of my closed loop system. As you can see, initially it takes large jumps before making some small back and forth adjustments before reaching setpoint value, but even after setpoint is reached the motor still struggles to keep a constant steady-state speed.



Figure 16: DC motor output results from serial console

To summarize my results, I will link my results to the requirements as shown in table 4.

Steady-state speed	Partly fulfilled
Steady-state error	Fulfilled
Overshoot	Sometimes fulfilled
Rising time	Not fulfilled
Stability	Fulfilled
Digital controller	Fulfilled

Table 4: Validation of requirements

Steady-state speed is partly fulfilled because the DC motor can reach the desired speed, but its not able to keep it constant without minor offsets within the $\pm 10\%$ steady state error. Overshoot is only fulfilled sometimes; the system behaviour is a bit random for each time I run the code. Rising time is not fulfilled, it takes longer than 10 seconds. The system is stable insofar as it reaches the setpoint value and stay there with $\pm 10\%$ variances. The control loop system is implemented as a digital controller.

Conclusions

Overall, the system functionality works as intended with some minor flaws that resulted in not fulfilling all the requirements. The benefit of using a PID-library is that it saves you time, the downside is that it gives you less control of the system behaviour. I was using the right PID parameters and sampling time, but still the system didn't fully match the behaviour of my Simulink simulation. When it comes to rising time, it might be that the DC motor had a higher static friction than expected, which resulted in the PID-controller having a harder time to get the initial rotation. Another issue might be that my speed-reading algorithm might be incorrect, so that the measured rpm is not equal to true rpm which again gives wrong inputs to the PID controller. When it comes to the varying steady-state speed after setpoint is reached, I think is because of the sampling rates or the PID-library itself, it seems to be more stable with faster sampling rates. The random overshoots I suspect might also be in connection with static and dynamic friction or disturbances in the system itself.

Appendix: Source codes

1. Main program

```
#include <PID_v1.h> // PID library

#define interruptPin 21
#define enA 8
#define in1 3
#define in2 2

unsigned long timestamp = 0L; // Timestamp initializer
double pulses = 0; // Pulse count initializer
double Setpoint, Input, Output; // Setpoint is wanted speed, Input is
measured rpm from sensor, Output is PWM signal to DC motor
double prev_pulses = 0; // Saves pulsecount from previous rpm sample
double Kp=0.0012927, Ki=0.0051706, Kd=0; // PID parameters

PID myPID(&Input, &Output, &Setpoint, Kp, Ki, Kd, DIRECT); // Uses defined
variables as arguments to PID controller library function

void count() { // Interrupt function routine for pulse counting
    pulses++;
}

void setup() {

    pinMode(interruptPin, INPUT);
    pinMode(enA, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
```

```

    attachInterrupt(digitalPinToInterrupt(interruptPin), count, RISING); //
Interrupt routine that counts every rising pulse from sensor

    digitalWrite(in1, LOW); // Set initial rotation direction
    digitalWrite(in2, HIGH);

    Serial.begin(9600);
    Setpoint = 500; // Defines wanted rpm
    myPID.SetMode(AUTOMATIC); // PID mode set to automatic so it
automatically performs speed adjustments

}

void measureRPM(){ // Function that reads a sample of rpm every 0.5 second
from sensor

    if (millis () - timestamp >= 500) // triggered every half second
    {
        noInterrupts(); // No interrupts while taking speed sample
        timestamp += 500 ; // setup for next time
        double now_pulses = pulses; // sample the count
        double count = now_pulses - prev_pulses; // take difference from last
sample
        prev_pulses = now_pulses ; // update the previous value
        double rpm = count*120/1024; // convert pulses counted in a 0.5 s
timeframe to rotations per minute
        Input = rpm; // Declare measured speed as input to PID controller
        Serial.print("RPM:");
        Serial.println(Input);
        interrupts(); // Turn interrupts on again after taking speed sample
    }
}

void PWMout(){ // PWM function
    analogWrite(enA, Output); // Send PWM signal to L298N Enable pin
}

void loop() { // Main loop function

    measureRPM(); // Measure rpm
    myPID.Compute(); // Compare speed to Setpoint and makes speed adjustment
computation
    PWMout(); // Sends PWM voltage to DC motor that is proportional to the
speed computation from PID controller

}

```

2. PID library

2.1 Header file

```
#ifndef PID_v1_h
#define PID_v1_h
#define LIBRARY_VERSION 1.2.1

class PID
{

public:

    //Constants used in some of the functions below
    #define AUTOMATIC 1
    #define MANUAL 0
    #define DIRECT 0
    #define REVERSE 1
    #define P_ON_M 0
    #define P_ON_E 1

    //commonly used functions
    *****
    PID(double*, double*, double*,          // * constructor. links the PID
to the Input, Output, and
        double, double, double, int, int); // Setpoint. Initial tuning
parameters are also set here.
                                                    // (overload for specifying
proportional mode)

    PID(double*, double*, double*,          // * constructor. links the PID
to the Input, Output, and
        double, double, double, int);      // Setpoint. Initial tuning
parameters are also set here

    void SetMode(int Mode);                  // * sets PID to either Manual
(0) or Auto (non-0)

    bool Compute();                          // * performs the PID
calculation. it should be
                                                    // called every time loop()
cycles. ON/OFF and
                                                    // calculation frequency can be
set using SetMode
                                                    // SetSampleTime respectively

    void SetOutputLimits(double, double);    // * clamps the output to a
specific range. 0-255 by default, but
                                                    // it's likely the user will
want to change this depending on
                                                    // the application

    //available but not commonly used functions
    *****
    void SetTunings(double, double,         // * While most users will set
the tunings once in the
        double);                          // constructor, this function
gives the user the option
```

```

// of changing tunings during
runtime for Adaptive control
    void SetTunings(double, double, // * overload for specifying
proportional mode          double, int);

    void SetControllerDirection(int); // * Sets the Direction, or "Action"
of the controller. DIRECT
// means the output will increase when error is
positive. REVERSE
// means the opposite. it's very unlikely that
this will be needed
// once it is set in the constructor.
    void SetSampleTime(int); // * sets the frequency, in
Milliseconds, with which
// the PID calculation is
performed. default is 100

//Display functions
*****
double GetKp(); // These functions query the pid for interal
values.
double GetKi(); // they were created mainly for the pid
front-end,
double GetKd(); // where it's important to know what is
actually
int GetMode(); // inside the PID.
int GetDirection(); //

private:
void Initialize();

double dispKp; // * we'll hold on to the tuning parameters in
user-entered
double dispKi; // format for display purposes
double dispKd; //

double kp; // * (P)roportional Tuning Parameter
double ki; // * (I)ntegral Tuning Parameter
double kd; // * (D)erivative Tuning Parameter

int controllerDirection;
int pOn;

double *myInput; // * Pointers to the Input, Output, and
Setpoint variables
double *myOutput; // This creates a hard link between the
variables and the
double *mySetpoint; // PID, freeing the user from having to
constantly tell us
// what these values are. with
pointers we'll just know.

unsigned long lastTime;
double outputSum, lastInput;

unsigned long SampleTime;
double outMin, outMax;
bool inAuto, pOnE;

```

```
};
#endif
```

2.2 Source file

```

/*****
*****
* Arduino PID Library - Version 1.2.1
* by Brett Beauregard <br3ttb@gmail.com> brettbeauregard.com
*
* This Library is licensed under the MIT License
*****
*****/

#ifdef ARDUINO >= 100
  #include "Arduino.h"
#else
  #include "WProgram.h"
#endif

#include <PID_v1.h>

/*Constructor
(...) *****/
* The parameters specified here are those for for which we can't set up
* reliable defaults, so we need to have the user set them.
*****/
/
PID::PID(double* Input, double* Output, double* Setpoint,
         double Kp, double Ki, double Kd, int POn, int ControllerDirection)
{
  myOutput = Output;
  myInput = Input;
  mySetpoint = Setpoint;
  inAuto = false;

  PID::SetOutputLimits(0, 255);          //default output limit corresponds
to                                         //the arduino pwm limits

  SampleTime = 500;                      //default Controller Sample Time is 0.1
seconds

  PID::SetControllerDirection(ControllerDirection);
  PID::SetTunings(Kp, Ki, Kd, POn);

  lastTime = millis()-SampleTime;
}

/*Constructor
(...) *****/
* To allow backwards compatability for v1.1, or for people that just
want
* to use Proportional on Error without explicitly saying so
*****/
/

```

```

PID::PID(double* Input, double* Output, double* Setpoint,
         double Kp, double Ki, double Kd, int ControllerDirection)
:PID::PID(Input, Output, Setpoint, Kp, Ki, Kd, P_ON_E,
ControllerDirection)
{

}

/* Compute()
*****
*      This, as they say, is where the magic happens.  this function should
be called
*      every time "void loop()" executes.  the function will decide for
itself whether a new
*      pid Output needs to be computed.  returns true when the output is
computed,
*      false when nothing has been done.
*****
*****/
bool PID::Compute()
{
    if(!inAuto) return false;
    unsigned long now = millis();
    unsigned long timeChange = (now - lastTime);
    if(timeChange>=SampleTime)
    {
        /*Compute all the working error variables*/
        double input = *myInput;
        double error = *mySetpoint - input;
        double dInput = (input - lastInput);
        outputSum+= (ki * error);

        /*Add Proportional on Measurement, if P_ON_M is specified*/
        if(!pOnE) outputSum-= kp * dInput;

        if(outputSum > outMax) outputSum= outMax;
        else if(outputSum < outMin) outputSum= outMin;

        /*Add Proportional on Error, if P_ON_E is specified*/
        double output;
        if(pOnE) output = kp * error;
        else output = 0;

        /*Compute Rest of PID Output*/
        output += outputSum - kd * dInput;

        if(output > outMax) output = outMax;
        else if(output < outMin) output = outMin;
        *myOutput = output;

        /*Remember some variables for next time*/
        lastInput = input;
        lastTime = now;
        return true;
    }
    else return false;
}

```



```

/*
SetTunings(...)*****
*
* This function allows the controller's dynamic performance to be
adjusted.
* it's called automatically from the constructor, but tunings can also
* be adjusted on the fly during normal operation
*****
***/
void PID::SetTunings(double Kp, double Ki, double Kd, int POn)
{
    if (Kp<0 || Ki<0 || Kd<0) return;

    pOn = POn;
    pOnE = POn == P_ON_E;

    dispKp = Kp; dispKi = Ki; dispKd = Kd;

    double SampleTimeInSec = ((double)SampleTime)/1000;
    kp = Kp;
    ki = Ki * SampleTimeInSec;
    kd = Kd / SampleTimeInSec;

    if(controllerDirection ==REVERSE)
    {
        kp = (0 - kp);
        ki = (0 - ki);
        kd = (0 - kd);
    }
}

/*
SetTunings(...)*****
*
* Set Tunings using the last-rembered POn setting
*****
***/
void PID::SetTunings(double Kp, double Ki, double Kd){
    SetTunings(Kp, Ki, Kd, pOn);
}

/* SetSampleTime(...)
*****
* sets the period, in Milliseconds, at which the calculation is performed
*****
***/
void PID::SetSampleTime(int NewSampleTime)
{
    if (NewSampleTime > 0)
    {
        double ratio = (double)NewSampleTime
                        / (double)SampleTime;
        ki *= ratio;
        kd /= ratio;
        SampleTime = (unsigned long)NewSampleTime;
    }
}

```

```

/* SetOutputLimits(...)*****
 *      This function will be used far more often than
SetInputLimits.  while
 *      the input to the controller will generally be in the 0-1023 range
(which is
 *      the default already,)  the output will be a little different.  maybe
they'll
 *      be doing a time window and will need 0-8000 or something.  or maybe
they'll
 *      want to clamp it from 0-125.  who knows.  at any rate, that can all be
done
 *      here.

*****/
void PID::SetOutputLimits(double Min, double Max)
{
    if(Min >= Max) return;
    outMin = Min;
    outMax = Max;

    if(inAuto)
    {
        if(*myOutput > outMax) *myOutput = outMax;
        else if(*myOutput < outMin) *myOutput = outMin;

        if(outputSum > outMax) outputSum= outMax;
        else if(outputSum < outMin) outputSum= outMin;
    }
}

/*
SetMode(...)*****
 *
 *      Allows the controller Mode to be set to manual (0) or Automatic (non-
zero)
 *      when the transition from manual to auto occurs, the controller is
 *      automatically initialized

*****
***/
void PID::SetMode(int Mode)
{
    bool newAuto = (Mode == AUTOMATIC);
    if(newAuto && !inAuto)
    { /*we just went from manual to auto*/
        PID::Initialize();
    }
    inAuto = newAuto;
}

/*
Initialize()*****
 *
 *      does all the things that need to happen to ensure a bumpless transfer
 *      from manual to automatic mode.

*****
***/
void PID::Initialize()
{
    outputSum = *myOutput;

```

```

    lastInput = *myInput;
    if(outputSum > outMax) outputSum = outMax;
    else if(outputSum < outMin) outputSum = outMin;
}

/*
SetControllerDirection(...)*****
*
* The PID will either be connected to a DIRECT acting process (+Output
leads
* to +Input) or a REVERSE acting process(+Output leads to -Input.) we
need to
* know which one, because otherwise we may increase the output when we
should
* be decreasing. This is called from the constructor.

*****
***/
void PID::SetControllerDirection(int Direction)
{
    if(inAuto && Direction !=controllerDirection)
    {
        kp = (0 - kp);
        ki = (0 - ki);
        kd = (0 - kd);
    }
    controllerDirection = Direction;
}

/* Status
Functions*****
* Just because you set the Kp=-1 doesn't mean it actually happened. these
* functions query the internal state of the PID. they're here for display
* purposes. this are the functions the PID Front-end uses for example

*****
***/
double PID::GetKp(){ return dispKp; }
double PID::GetKi(){ return dispKi;}
double PID::GetKd(){ return dispKd;}
int PID::GetMode(){ return inAuto ? AUTOMATIC : MANUAL;}
int PID::GetDirection(){ return controllerDirection;}

```

Source: <https://playground.arduino.cc/Code/PIDLibrary/>